

Prolog Schedule Assignment Report - 23324986 Petra Sartori

Declaration of Academic Integrity

NAME:	Petra Sartori
STUDENT ID NUMBER	23324986
PROGRAMME	Computer Science
MODULE CODE	CSC1018
ASSIGNMENT TITLE	Fixture Scheduler
SUBMISSION DATE	13th April 2026

I understand that the University regards breaches of academic integrity and plagiarism as grave and serious.

I have read and understood the DCU Academic Integrity and Plagiarism Policy. I accept the penalties that may be imposed should I engage in practice or practices that breach this policy.

I have identified and included the source of all facts, ideas, opinions and viewpoints of others in the assignment references. Direct quotations, paraphrasing, discussion of ideas from books, journal articles, internet sources, module text, or any other source whatsoever are acknowledged and the sources cited are identified in the assignment references.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my work.

I have used the DCU library referencing guidelines (available at https://www4.dcu.ie/library/classes_and_tutorials/citingreferencing.shtml **and/or** the appropriate referencing system recommended in the assignment guidelines and/or programme documentation.

By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

By signing this form or by submitting material for assessment online I confirm that I have read and understood the DCU Academic Integrity and Plagiarism Policy (available at <http://www.dcu.ie/registry/examinations/index.shtml>).

Signature Petra Sartori

Date: 12th April 2026

Research Sources

*I referred to the following sources mostly to help with understanding the CLP(FD) library that I used to solve this assignment. I looked at examples of its use in combinatorial constraints problems like magic sequence (mostly for use of global_cardinality) and the social golfer problem. These examples helped me to come up with a structure for my program. I referred to user manuals for CLP(FD) arithmetic constraints I used to apply all constraints. I referred to The Power of Prolog website and YouTube videos for general research on Prolog. It helped me with syntax and logical understanding while I was working on this assignment.

SWI-Prolog manual. (2026). Swi-Prolog.org.
<https://www.swi-prolog.org/man/clpfd.html>

The Power of Prolog. (2025). Metalevel.at. <https://www.metalevel.at/prolog>

Combinatorial Constraints - SICStus Prolog. (2026). Sics.se.
<https://sicstus.sics.se/sicstus/docs/4.0.5/html/sicstus/Combinatorial-Constraints.html>

SICStus Prolog: FDBG An Example Session. (2026). Sics.se.
<https://sicstus.sics.se/sicstus/docs/4.3.3/html/sicstus/FDBG-An-Example-Session.html>

The Social Golfer Problem. (2026). Metalevel.at. <https://www.metalevel.at/sgp/>

Design and implementation of my Fixed Scheduler

The implementation of my program was done incrementally, starting from basic facts that split 30 teams into groups of 5. All required fixtures are derived from these facts, making sure that every team plays against every other team in a particular group only once. I imported and used the CLP(FD) (Constraint Logic Programming over Finite Domains) library in multiple predicates for this task because I have found it to be the most elegant solution for solving combinatorial problems such as this one. Most of the examples I have seen using this approach dealt with planning, scheduling and allocation so I have concluded that it would be the most appropriate way to enforce arithmetic constraints mentioned below in greater detail.

Each fixture(game) is in the format *game(TeamA, TeamB, Day, Home)*. Day represents match day, and Home is a binary flag variable which tells us whether the team is playing home or away. The program builds a list of games, extracts lists of Day and Home variables, and applies CLP(FD) constraints to apply schedule rules. Predicate *labeling* that is included in CLP(FD) library assigns values to these variables, which produces a valid schedule.

Predicate *pair* is a helper predicate used by *games*. It pairs a specific team with every other team in their respective group. The base case occurs when there are no teams left to pair. The recursive case generates a game term, but fills out only Team A and Team B arguments, while Day and Home remain anonymous for now. It recursively calls itself to pair up the same team with the rest of the teams in the group.

Predicate *games* generates all games for one group of all teams. The base case occurs when there are no teams left, so the result is an empty list. The recursive case takes the first team and pairs it with every other team in the

group by calling the helper predicate *pair*. It calls itself recursively on the tail to generate all remaining games. All games for that group are appended to the output list. The difference between *pair* and *games* is that *pair* generates fixtures for one specific team, while *games* calls it along with the recursive call to generate games for all teams in one group.

Predicate *all_games(Games)* is another extension of the same logic, but it uses *findall* to iterate through every group to generate a full list of games for the entire competition. It extracts the team list for each group and calls *games* to generate fixtures for each group, which creates a nested list for each. These nested lists are appended to one final Games list which contains combinations across all groups.

Example usage: *all_games* returns a correct number of games - 60

```
2 ?- all_games(G), length(G, N).
G = [game(t1, t2, _, _), game(t1, t3, _, _), game(t1, t4, _, _), game(t1, t5, _, _), game(t2, t3, _, _),
game(t2, t4, _, _), game(t2, t5, _, _), game(t3, t4, _, _), game(..., ..., ..., ...)|...],
N = 60.
```

Example usage: showing that each team has 4 fixtures

```
3 ?- all_games(G), team_days(t1, G, Ds), length(Ds, N).
G = [game(t1, t2, _A, _), game(t1, t3, _B, _), game(t1, t4, _C, _), game(t1, t5, _D, _), game(t2, t3, _A, _),
game(t2, t4, _B, _), game(t2, t5, _C, _), game(t3, t4, _D, _), game(..., ..., ..., ...)|...],
Ds = [_A, _B, _C, _D],
N = 4.
```

Predicate *days* acquires all match-day variables from the list of games. The base case occurs when the game list is empty, and a recursive case matches the first game, takes its Day argument and adds it to the output list. It recursively calls itself to do the same for the rest of the games. This list is needed because it is used in satisfying constraints.

schedule predicate is the main predicate of the program, it brings all of the implementation together by calling all of the helper functions and constraints the domains of Days and Homes. Firstly, it calls *all_games* to produce a complete list of all games that will be played. It calls *days* and *homes* to extract day and home variables from each game. Days variables are constrained to be between 1 and 40, and Homes must be either 0 or 1. Next, it calls three predicates which enforce three main constraints (they are explained in greater detail below): *three_daily* makes sure that no more than three matches are held on the same day, *rest_days* makes sure that each

team has at least four days of rest between games, and *home_away* makes sure that each team has the same number of home and away games. Lastly, it uses *append* to combine Days and Homes variables into one list, and then calls *labeling*, which assigns a value to each variable in the list we created. This produces a full schedule. The last touch I added to the implementation of *schedule* was importing *library(random)* and calling *random_permutation* on Days and Homes, and then passing them to *labeling*. I did this because originally, after I clicked ; to produce more schedules, they would look almost exactly the same, except for one variable change, which was caused by backtracking. By randomizing Days and Homes, the schedules still maintain all constraints, but they do not appear the exact same.

Example usage: *schedule(G)* generates a full valid schedule. After pressing ; we get another valid schedule

```
5 ?- schedule(G).
G = [game(t1,t2,4,0),game(t1,t3,26,0),game(t1,t4,16,1),game(t1,t5,11,1),game(t2,t3,21,0),game(t2,t4,26,0),
game(t2,t5,16,1),game(t3,t4,11,0),game(t3,t5,6,0),game(t4,t5,1,0),game(t6,t7,18,1),game(t6,t8,7,1),game(
t6,t9,2,0),game(t6,t10,23,0),game(t7,t8,13,0),game(t7,t9,8,1),game(t7,t10,28,1),game(t8,t9,23,1),game(t8,
t10,18,0),game(t9,t10,13,1),game(t11,t12,27,0),game(t11,t13,17,0),game(t11,t14,3,1),game(t11,t15,22,1),ga
me(t12,t13,7,1),game(t12,t14,12,0),game(t12,t15,2,0),game(t13,t14,22,0),game(t13,t15,12,1),game(t14,t15,1
7,0),game(t16,t17,29,0),game(t16,t18,17,1),game(t16,t19,3,1),game(t16,t20,24,0),game(t17,t18,7,0),game(t1
7,t19,14,1),game(t17,t20,2,0),game(t18,t19,22,0),game(t18,t20,12,1),game(t19,t20,29,1),game(t21,t22,23,0),
game(t21,t23,14,0),game(t21,t24,28,1),game(t21,t25,8,1),game(t22,t23,28,0),game(t22,t24,33,0),game(t22,t
25,18,1),game(t23,t24,6,0),game(t23,t25,1,0),game(t24,t25,13,0),game(t26,t27,1,0),game(t26,t28,8,1),game(
t26,t29,14,1),game(t26,t30,19,0),game(t27,t28,16,0),game(t27,t29,6,0),game(t27,t30,11,1),game(t28,t29,21,
0),game(t28,t30,3,1),game(t29,t30,26,0)] ;
G = [game(t1,t2,4,0),game(t1,t3,26,0),game(t1,t4,16,1),game(t1,t5,11,1),game(t2,t3,21,0),game(t2,t4,26,0),
game(t2,t5,16,1),game(t3,t4,11,0),game(t3,t5,6,0),game(t4,t5,1,0),game(t6,t7,18,1),game(t6,t8,7,0),game(
t6,t9,2,1),game(t6,t10,23,0),game(t7,t8,13,0),game(t7,t9,8,1),game(t7,t10,28,1),game(t8,t9,23,0),game(t8,
t10,18,0),game(t9,t10,13,1),game(t11,t12,27,0),game(t11,t13,17,0),game(t11,t14,3,1),game(t11,t15,22,1),ga
me(t12,t13,7,1),game(t12,t14,12,0),game(t12,t15,2,0),game(t13,t14,22,0),game(t13,t15,12,1),game(t14,t15,1
7,0),game(t16,t17,29,0),game(t16,t18,17,1),game(t16,t19,3,1),game(t16,t20,24,0),game(t17,t18,7,0),game(t1
7,t19,14,1),game(t17,t20,2,0),game(t18,t19,22,0),game(t18,t20,12,1),game(t19,t20,29,1),game(t21,t22,23,0),
game(t21,t23,14,0),game(t21,t24,28,1),game(t21,t25,8,1),game(t22,t23,28,0),game(t22,t24,33,0),game(t22,t
```

Implementation of constraints:

1. There are no more than three matches on any day

three_daily predicate calls helper predicates which bring all of the implementation together. It calls: *build_pairs*, *global_cardinality* and *three_max*. Since the Days list only contains a list of match-day variables, I had to find a way to link them with a count for each possible day and enforce the maximum of three games per day rule.

build_pairs(1, 40, Pairs) constructs a list of placeholders (Day-Count) which will keep track of how many matches appear on a particular day.

The base case happens when the current Day number gets greater than the Max allowed(40), and in that case we stop building the list and return an empty list.

The recursive case starts with Day - Count, so if Day is 1, the list starts with 1 - Count. Next, we check if we are still inside the specified range with Day =<

Max, and move on to the next Day with NextDay is Day + 1. Lastly we recursively call the function to do this for the rest of the list.

CLP(FD) - *global_cardinality(+Xs, +Vals)*

Xs - [X1, ..., Xd] is a list of integers or domain variables

Vals = [K1 - V1, ..., Kn - Vn] is a list of pairs where each key Ki is a unique integer and Vi is a domain variable or an integer. This function is not implemented manually, it comes with the CLP(FD) library.

In this case, Days are Xs, and Pairs are Vals. I used it to connect placeholders previously created with *build_pairs* to the actual day variables, so that each Count variable shows how many times does the particular day occur in the Days list.

three_max(Pairs) goes through the list of day-count pairs and recursively applies Count $\#=< 3$ CLP(FD) constraint to ensure that there are no more than three matches on any day in the list. $\#=</2$ is part of the CLP(FD) library and it enforces the constraint that the Expression 1 (Count) has to be less than or equal to Expression 2 (3).

2. Each team has at least four rest days between fixtures

Predicate *team_days/3* takes a team and a list of games, traverses the list of games, and collects the days of all games that the given team is playing in, compiling them into a new list. This new list is needed to keep track of all days on which each team plays, so we can apply a four rest day constraint later. The base case occurs when there are no games left. In that case there are no more days to collect, so the result is an empty list. The first recursive case occurs when we encounter a day which contains a game(A, B, Day) in which the target team is either A or B. I have done this using a semi deterministic *memberchk/2* predicate, which is true when a target element is an element of a particular list. We include that Day in the output list and recursively call the predicate again to continue until the end of the list. I used a cut to prevent unnecessary backtracking through the rest of the games. The second recursive case happens when we encounter a game(A, B, Day) which does not contain a target team as either A or B. All we have to do in that case is ignore that game and continue searching.

check_length predicate takes a day and checks whether the difference between the given day and the rest of the days in the list is equal to or greater than 5. I chose to check against 5 since the constraint states that there has to be at least four full rest days between fixtures. The check is done recursively

to check the given day against the entire Tail of the days list. It uses CLP(FD) library operator `#>=` where Expression 1 calculates the difference between two days, and Expression 2 is 5. Note: this predicate only checks the length between one specific day and the rest of days, not all pairs.

rest predicate calls on *check_length* predicate recursively to check the length of days in between fixtures for the whole list of days. This way all pairs are covered because the process is repeated for each day in the list and all pairs are checked only once.

all_teams predicate returns a list of all teams using *findall/3*.

all_teams_rest predicate firstly calls *team_days* to get days of all games for a team, then the *rest* predicate to check the gap between those days, and then it recursively calls itself again to repeat the process for all teams.

Rest_days predicate just calls helper predicates *all_teams(Teams)* and *all_teams_rest(Teams, Games)* to bring the implementation of the constraint together.

3. Each team has the same number of home and away fixtures in the whole schedule

Initially, I had no variable that indicates whether the game is home or away for a particular team, so the *game* predicate was of a form *game(Team, Opponent, Day)*. To implement this constraint, I introduced a new binary variable *Home*. I used it in such a way that when Home is 0, Team A is home, otherwise Team B is home, and Team A is away. I went through my code and added a new anonymous variable which represents Home for each time I used the *game* predicate.

home_flags predicate traverses the list of games and collects a list of home/away flags for one team. There are three recursive cases. Since we pass *game(A, B, _, Home)*, case 1 occurs when the target team is team A, and the Flag is defined as 1 - Home, so it is set to 1 when team A plays at home, and 0 when team A is playing away. Case 2 occurs when the target team is team B, and the Flag is defined as Home, which means that it is set to 1 when B is playing at home, and to 0 when they are playing away. In both cases I used a cut after the team match to prevent backtracking. Case 3 occurs when the target team is not in the current game, so that game is ignored and we continue traversing. Anytime we encounter a game our target team is in, the computed flag is added to the flag list.

After we collected a list of home flags, we have to ensure that there are 2 home matches per team. *all_teams_home(Team, Games)* calls *home_flags* to obtain all flags for a team and checks whether the sum of them is 2 (since there is supposed to be two home flags with value 1, and two with value 0). I also used *#=* arithmetic constraint from the CLP(FD) library to enforce this rule.

home_away(Games) predicate calls for *all_teams(Teams)* to obtain a list of all teams and *all_teams_home(Teams, Games)* to ensure every team plays 2 home matches.

homes(Games, Homes) is called from the *schedule(G)* predicate, it extracts all of the Home variables from all games recursively before we constrain *Homes* to 0..1 in *schedule*. Predicate *homes* does not actually calculate the value of home flags like *home_flags* predicate does, they are just used for the purpose of constraining them in *schedule*.

Implementation challenges:

While the final version of my program can generate valid schedules that enforce all constraints mentioned in assignment specifications, I could not optimize the schedule length to be as short as possible because it significantly impacted the time it took for the schedule to generate (search time would take minutes instead of seconds, or the schedule would not generate at all within reasonable time). I explored various ways to optimise this aspect of the assignment using various helper predicates to find the upper bound limit on all scheduled fixtures dynamically, to reduce the day search range, to minimize the final day used, etc. This would unfortunately affect the performance of the entire program as it takes too long to find a schedule that is both valid and the shortest possible. While importing random library to randomize the output of all schedules so they do not look the same, constraints are restricting the search space too much, so schedules are not as randomized as I hoped. Nevertheless, they are all valid and different.